

Applied Software Project Report

By

Amol Brahme

A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment of the requirements for the degree of Master of Science in Computer Science

Oct, 2026



Scaler Mentee Email ID: amolsbrahme@gmail.com
Thesis Supervisor: Naman Bhalla
Date of Submission: 11/10/2026

© The project report of Amol Brahme is approved, and it is acceptable in quality and form for publication electronically

Certification

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of the Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

Project Guide / Supervisor

Declaration

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from 4 Dec 2023 to Nov 20, 2024 is the result of my own individual endeavor. The project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. I used Claude (Claude Code), an AI coding assistant by Anthropic, to assist with coding, debugging, refactoring, documentation, diagram generation, and drafting portions of this report, and all such AI-assisted work was reviewed, verified, and is owned by me. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Amol Brahme

Date: 10 Oct 2026

Acknowledgment

I would like to express my sincere gratitude to my mentor and thesis supervisor, Naman Bhalla, for his guidance, feedback, and support throughout the duration of this project. I am also thankful to the mentors, teaching assistants, and the broader Scaler Neovarsity community for their continuous encouragement and for creating an environment that made this applied project possible. Finally, I would like to thank my family and friends for their patience, understanding, and support throughout the course of this Master's program.

Table of Contents

Table of Contents

List of Tables 7
List of Figures 8
Applied Software Project 9
Abstract 9
Chapter 1 — Project Description 11
1.1 Problem Statement 11
1.2 System Overview 11
1.3 Core Components 12
1.4 End-to-End Order Placement Workflow 13
Chapter 2 — Requirement Gathering 15
2.1 Requirement Analysis 15
2.2 Functional Requirements 15
2.3 Non-Functional Requirements 16
2.4 Use Case Analysis 17
2.5 Feature Set 18
Chapter 3 — System Design 20
3.1 Overview of the Low-Level Design 20
3.2 Order Service Domain Model 20
Chapter 4 — Database Schema Design 22
4.1 Overview 22
4.2 Schema 22
4.3 Entities 22
4.4 Relationships 23
Chapter 5 — Feature Development Process 25
5.1 Introduction 25
5.2 Request Flow 25
5.3 Resilience Engineering 25
5.4 Saga-Lite Compensation 26

Chapter 6 — Deployment Flow 27
6.1 Deployment Overview 27
6.2 Container Topology 27
6.3 Health Checks and Service Discovery 28
6.4 Cloud Deployment Path (Future Scope) 28
Chapter 7 — Technologies Used 30
7.1 Technology Stack Overview 30
7.2 Spring Boot and Spring Cloud 30
7.3 Keycloak and OAuth2/OIDC 30
7.4 MongoDB 30
7.5 MySQL and Flyway 30
7.6 Apache Kafka 30
7.7 Resilience4j 31
7.8 Spring 6.1 HTTP Interface Clients (RestClient) 31
7.9 Docker Compose and Testcontainers 31
7.10 OpenAPI / Swagger 31
7.11 Technology Summary 31
Chapter 8 — Conclusion 33
8.1 Conclusion 33
8.2 Key Takeaways 33
8.3 Practical Applications 33
8.4 Limitations 34
8.5 Future Scope 34
References 36

List of Tables

Table No.	Title	Page No.
2.01	Functional Requirements	15
2.02	Non-Functional Requirements	16
2.03	Feature Set	18
4.01	Foreign Key Relationships Across the Order and Inventory Schemas	23
4.02	Cardinality of Relationships	23
7.01	Technology Stack Summary	31

List of Figures

Figure No.	Title	Page No.
1.01	High-Level Microservices Architecture	12
1.02	End-to-End Order Placement Workflow	14
2.01	Use Case Diagram	18
3.01	Class Diagram – Order Service Domain Model	21
4.01	Entity-Relationship Diagram – Order and Inventory Schemas	24
6.01	Deployment Architecture (Docker Compose)	28

Applied Software Project

Abstract

This project presents the design and implementation of a Distributed E-Commerce Order Management System, a backend platform built using a microservices architecture with Spring Boot and Spring Cloud. The system is composed of six independently deployable services – Product, Order, Inventory, Notification, an API Gateway, and a Eureka-based Discovery Server – each owning its own datastore and communicating over well-defined REST and event-driven interfaces. Placing an order synchronously reserves stock from the Inventory Service through a RestClient-backed HTTP call guarded by a Resilience4j circuit breaker and retry policy, and, on success, asynchronously notifies the customer through a Kafka event that the Notification Service consumes. A lightweight compensation mechanism releases any stock already reserved for an order if a later line item cannot be reserved, keeping the system consistent without the overhead of a full saga-orchestration framework.

Every request is authenticated and authorized using Keycloak (OAuth2/OIDC) at the API Gateway, and each service persists to its own datastore – MongoDB for the product catalog, and separate MySQL schemas, versioned with Flyway, for orders and inventory – following the database-per-service pattern. The complete stack is containerized and orchestrated locally using Docker Compose, with unit and Testcontainers-backed integration tests covering each service.

This project is relevant to the e-commerce and retail-technology industry, where high-traffic platforms require order-processing systems that remain available and consistent even when individual downstream dependencies, such as inventory or payment services, are slow or temporarily unavailable. The patterns demonstrated here – service decomposition, resilience engineering (circuit breakers, retries, fallback and compensation logic), event-driven notification, and centralized authentication – are directly transferable to production order-management, warehouse, and fulfillment systems across industries that need to scale beyond a single monolithic application.

Keywords:

Microservices, Spring Boot, Spring Cloud, Resilience4j, Circuit Breaker, Apache Kafka, Keycloak, OAuth2, Docker Compose, Database-per-Service.

Chapter 1 — Project Description

1.1 Problem Statement

Online retail platforms must process orders reliably even though the services an order depends on – stock levels, pricing, customer notification – are themselves separate, independently evolving systems. A monolithic order-processing module makes this difficult: a bug or slow query in one area of functionality can block unrelated functionality, the whole application must be redeployed for any change, and the application cannot be scaled selectively for the parts of it that are actually under load (for example, checkout traffic during a sale). This project addresses that problem by decomposing order management into independently deployable microservices, each owning a single business capability and its own data, and by explicitly engineering for the failure modes that decomposition introduces – namely, that a downstream service call across the network can be slow, can fail outright, or can legitimately reject the request (for example, insufficient stock) rather than fail.

1.2 System Overview

The system models a distributed e-commerce order pipeline as six independently deployable Spring Boot services, fronted by a Spring Cloud Gateway and backed by a Eureka service registry. Placing an order synchronously reserves stock from the Inventory Service and, on success, asynchronously notifies the customer through a Kafka event. Every request is authenticated and authorized with Keycloak (OAuth2/OIDC) at the gateway, and each service owns its own datastore, following the database-per-service pattern. The whole stack is containerized and orchestrated locally with Docker Compose.

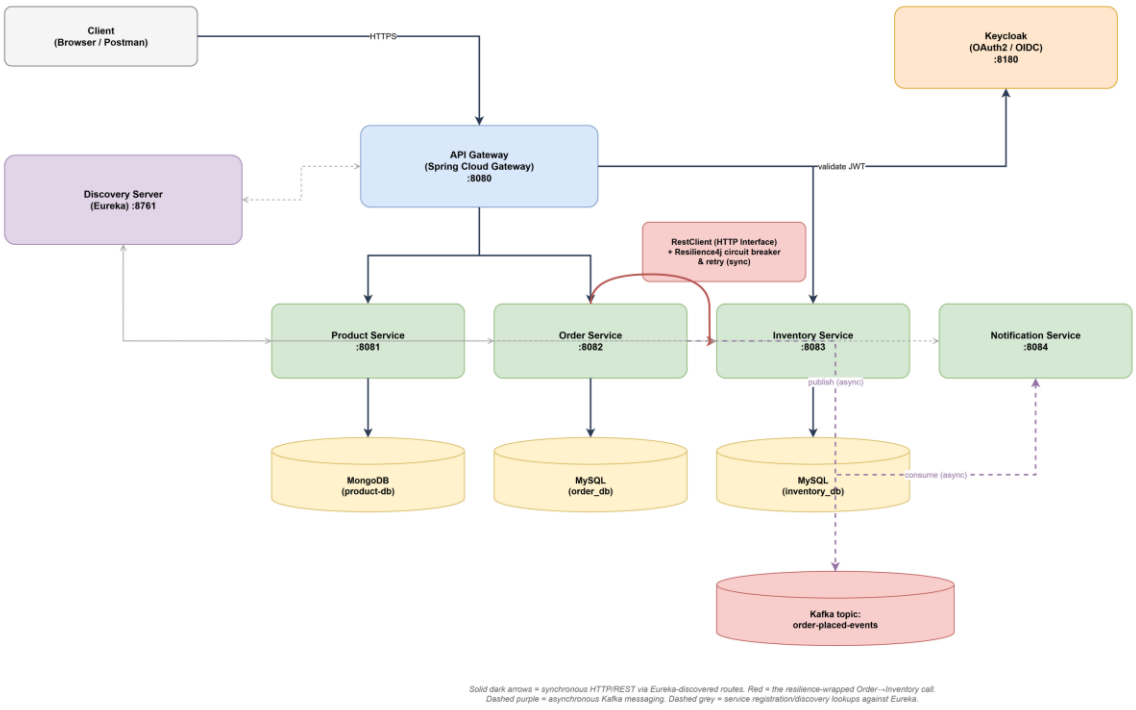


Figure 1.01: High-Level Microservices Architecture

1.3 Core Components

The platform is composed of the following six services, each with a single, well-defined responsibility:

- **Discovery Server (port 8761):** a Netflix Eureka registry that every other service registers with on startup and queries at call time, so services can locate one another by logical name instead of hard-coded host/port pairs.
- **API Gateway (port 8080):** a Spring Cloud Gateway instance that is the single entry point for client traffic, routes requests to the correct backend service via Eureka, and acts as an OAuth2 resource server that validates Keycloak-issued JWT access tokens before a request is allowed to reach a backend service.
- **Product Service (port 8081):** owns the product catalog, persisted in MongoDB. It exposes CRUD endpoints for products, keyed internally by a generated id and externally by a unique, indexed SKU.
- **Order Service (port 8082):** owns order placement and retrieval, persisted in a dedicated MySQL schema (order_db) versioned with Flyway. It orchestrates stock reservation

against the Inventory Service through a resilience-wrapped HTTP client and, on success, publishes an order-placed event to Kafka.

- **Inventory Service (port 8083):** owns stock levels per SKU, persisted in a dedicated MySQL schema (inventory_db) versioned with Flyway. It exposes endpoints to reserve, release, and replenish stock, using an optimistic-locking version column to guard against concurrent updates to the same SKU.
- **Notification Service (port 8084):** a Kafka consumer that listens on the order-placed-events topic and notifies the customer asynchronously once an order has been confirmed, decoupling notification delivery from the latency of the order-placement request itself.

1.4 End-to-End Order Placement Workflow

When a client submits an order, the request passes through the API Gateway, which validates the caller's JWT access token against Keycloak before forwarding the request to the Order Service. The Order Service then iterates over the requested line items and, for each one, calls the Inventory Service to reserve the requested quantity. This call is wrapped in a Resilience4j circuit breaker and retry policy so that transient Inventory Service failures are retried automatically and a persistently failing Inventory Service is detected and bypassed quickly rather than left to time out on every request. If every line item is reserved successfully, the order is persisted with status CONFIRMED and an order-placed event is published to Kafka, which the Notification Service consumes to notify the customer. If a line item cannot be reserved – either because of insufficient stock or because the Inventory Service is unavailable – the Order Service performs a "saga-lite" compensation step, releasing every item it had already reserved earlier in the same request, and persists the order with status FAILED and a recorded failure reason instead of publishing an event.

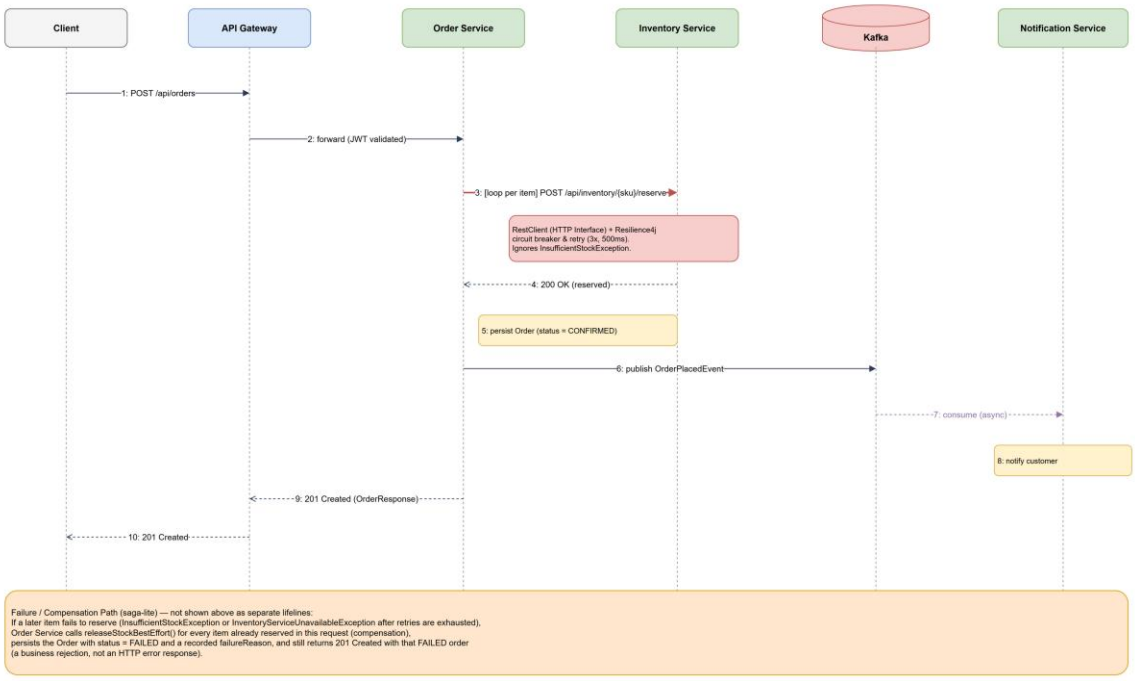


Figure 1.02: End-to-End Order Placement Workflow

Chapter 2 — Requirement Gathering

2.1 Requirement Analysis

Requirements were derived from the two user-facing roles the system serves: a customer who places and tracks orders, and an operations/admin actor who maintains the product catalog and stock levels. The functional requirements in Table 2.01 describe what the system must do; the non-functional requirements in Table 2.02 describe the operational qualities – resilience, consistency, security, observability – that the implementation was specifically engineered to satisfy, since these qualities, rather than the basic CRUD operations, are what a microservices decomposition puts at risk if not addressed deliberately.

2.2 Functional Requirements

Table 2.01: Functional Requirements

ID	Requirement Name	Description	Rationale	Module(s)
FR-01	Place Order	Customer submits an order with one or more line items (SKU + quantity).	Core business capability of the system.	Order Service
FR-02	Reserve Stock	Reserve the requested quantity for each line item before an order is confirmed.	Prevents overselling.	Order, Inventory
FR-03	Compensation on Partial Failure	Release already-reserved items if a later item in the same order cannot be reserved.	Keeps inventory counts consistent after a partial failure.	Order, Inventory
FR-04	Retrieve Order	Look up a single order by its order number.	Lets customers and support staff check order status.	Order Service
FR-05	List Customer Orders	List every order placed by a given customer id.	Supports order-history views.	Order Service

FR-06	Manage Product Catalog	Create, read, update, delete, and paginate products.	The catalog must be maintained over time.	Product Service
FR-07	Manage Stock Levels	Create a stock record for a new SKU and set its available quantity.	Lets operations staff replenish stock.	Inventory Service
FR-08	Notify Customer	Send a notification once an order has been confirmed.	Keeps the customer informed without blocking checkout.	Notification Service
FR-09	Authenticat e Requests	Every request to a backend service must carry a valid Keycloak-issued access token.	Protects the system from unauthorized access.	API Gateway
FR-10	Route and Discover Services	Route each incoming request to the correct service instance via the service registry.	Allows services to scale or move without reconfiguring clients.	Gateway, Discovery

2.3 Non-Functional Requirements

Table 2.02: Non-Functional Requirements

ID	Requirement	Implementation Approach	Module(s)
NF R-01	Availability	Inventory Service failures must not cascade into Order Service outages; handled with a Resilience4j circuit breaker and fallback.	Order Service
NF R-02	Fault Tolerance	Transient downstream failures are retried automatically: Resilience4j retry, 3 attempts, 500ms backoff.	Order Service
NF R-03	Data Consistency	Each service is the sole writer of its own data; cross-service consistency is kept via compensation rather than distributed transactions (database-per-service pattern).	All services
NF R-04	Security	All API traffic is authenticated and authorized before it reaches business logic, via a Keycloak OAuth2/OIDC resource server at the gateway.	API Gateway

NFR-05	Scalability	Services are stateless and independently deployable behind a registry-aware gateway, so any one of them can be scaled without affecting the others.	All services
NFR-06	Observability	The health of each service and its circuit breakers is inspectable at runtime via Spring Boot Actuator health, metrics, and circuitbreakers endpoints.	All services
NFR-07	Testability	Business logic is verifiable without a live external dependency, using unit tests plus Testcontainers-backed integration tests.	All services
NFR-08	Portability	The full stack runs identically on any machine with Docker installed, orchestrated with a single Docker Compose file.	All services

2.4 Use Case Analysis

The system supports two primary actors – the Customer and an Operations/Admin user – plus the system itself acting as a trigger for asynchronous notification. The principal use cases are:

- UC-01 Place an Order (Customer) – submit an order containing one or more SKUs and quantities.
- UC-02 Check Order Status (Customer) – retrieve an order by its order number.
- UC-03 View Order History (Customer) – list all orders previously placed by the customer.
- UC-04 Manage Product Catalog (Admin) – create, update, delete, and browse products.
- UC-05 Replenish Stock (Admin) – create a stock record for a SKU and set its available quantity.
- UC-06 Authenticate (Customer, Admin) – obtain and present a valid access token issued by Keycloak before any protected endpoint can be called.

• UC-07 Receive Order Notification (System → Customer) – triggered automatically by the order-placed Kafka event once UC-01 succeeds.

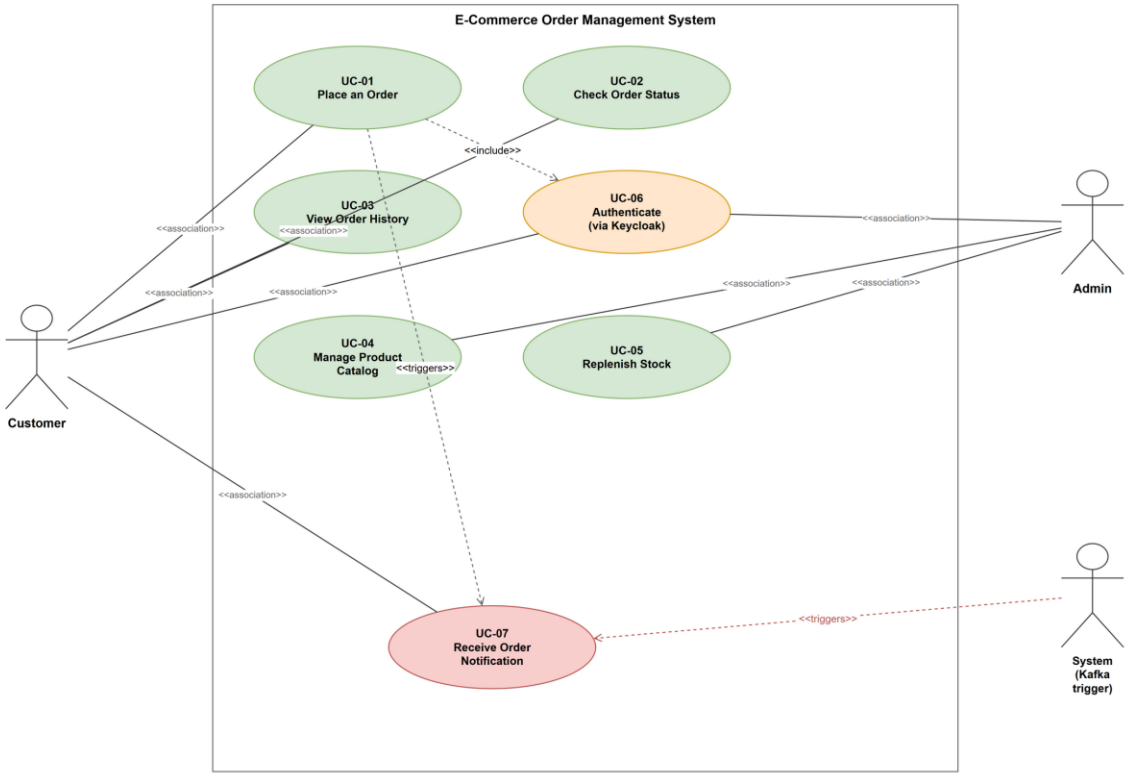


Figure 2.01: Use Case Diagram

2.5 Feature Set

Table 2.03 lists the concrete REST and event-driven features implemented against the requirements above, each mapped to the real API endpoint it corresponds to.

Table 2.03: Feature Set

ID	Feature	Description (Endpoint)	Module(s)
FS-01	Place Order	POST /api/orders – reserves stock synchronously, then publishes an order-placed event.	Order Service
FS-02	Get Order	GET /api/orders/{orderNumber}	Order Service
FS-03	List Orders by Customer	GET /api/orders?customerId=…	Order Service

FS-04	Create Product	POST /api/products	Product Service
FS-05	Get Product by Id / SKU	GET /api/products/{id}, GET /api/products/sku/{sku}	Product Service
FS-06	List Products	GET /api/products (paginated)	Product Service
FS-07	Update / Delete Product	PUT /api/products/{id}, DELETE /api/products/{id}	Product Service
FS-08	Create Stock Record	POST /api/inventory	Inventory Service
FS-09	Get Stock by SKU	GET /api/inventory/{sku}	Inventory Service
FS-10	Reserve / Release Stock	POST /api/inventory/{sku}/reserve, POST /api/inventory/{sku}/release	Inventory Service
FS-11	Set Available Quantity	PUT /api/inventory/{sku}	Inventory Service
FS-12	Consume Order-Placed Events	Kafka listener on order-placed-events notifies the customer.	Notification Service
FS-13	Authenticate via Keycloak	OAuth2/OIDC token issuance and JWT validation at the gateway.	Gateway, Keycloak

Chapter 3 — System Design

3.1 Overview of the Low-Level Design

Each service follows a conventional layered design: a REST controller exposes the service's API, a service layer holds the business logic, a repository layer (Spring Data JPA for the MySQL-backed services, Spring Data MongoDB for the Product Service) handles persistence, and a set of request/response DTOs decouple the public API shape from the internal domain model. The Order Service's domain model is the most involved, since it both owns its own persistent entities and depends on an outbound HTTP client to the Inventory Service, so it is used as the representative example below.

3.2 Order Service Domain Model

An *Order* aggregates one or more *OrderItem* entities (a one-to-many composition, cascading on delete) and carries an *OrderStatus* (CONFIRMED or FAILED, with a failure reason recorded for the latter). *OrderServiceImpl* is the central orchestrator: it depends on *OrderRepository* for persistence, *InventoryGateway* (an HTTP interface client, see Chapter 5) to reserve and release stock, and *OrderEventPublisher* to publish an *OrderPlacedEvent* to Kafka once an order is confirmed. Two custom checked business exceptions – *InsufficientStockException* and *InventoryServiceUnavailableException* – distinguish a legitimate business rejection from an infrastructure fault, which is the distinction the resilience configuration in Chapter 5 is built around.

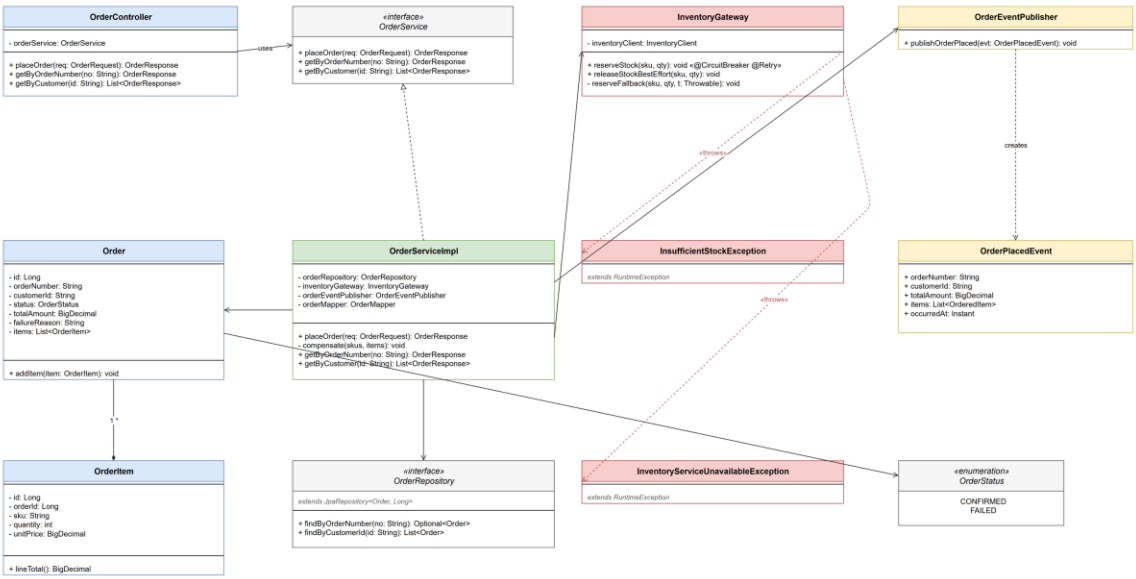


Figure 3.01: Class Diagram – Order Service Domain Model

Chapter 4 — Database Schema Design

4.1 Overview

The system follows the database-per-service pattern: no two services share a schema, and no service queries another service's database directly. The Product Service persists to MongoDB (document model, suited to a catalog whose attributes can vary by product category); the Order Service and Inventory Service each own a dedicated MySQL schema (order_db and inventory_db respectively), with every schema change captured as a versioned Flyway migration so the schema history is reproducible and auditable. Any data a service needs from another service's domain – for example, the Order Service needing a SKU's price at the moment an order is placed – is obtained by calling that service's API and, where appropriate, storing a denormalized snapshot rather than a live reference.

4.2 Schema

order_db (Order Service, MySQL, Flyway-managed):

- orders(id, order_number, customer_id, status, total_amount, failure_reason, created_at, updated_at)
- order_items(id, order_id, sku, quantity, unit_price)

inventory_db (Inventory Service, MySQL, Flyway-managed):

- stock(id, sku, available_quantity, reserved_quantity, version, created_at, updated_at)

Product Service (MongoDB, "products" collection):

- products(id, sku, name, description, category, price, initialStockHint, createdAt, updatedAt)

4.3 Entities

orders is the aggregate root for an order: order_number is generated and unique, status is CONFIRMED or FAILED, total_amount is a monetary DECIMAL(12,2), and failure_reason is populated only when status is FAILED. order_items holds a denormalized

sku and unit_price per line item – captured at the moment the order is placed so that a later catalog price change does not retroactively change a historical order – with a database-level CHECK constraint enforcing quantity > 0. stock holds, per SKU, the available_quantity that can still be sold and the reserved_quantity already committed to open orders, both guarded by non-negative CHECK constraints; a version column implements optimistic locking so that two concurrent reservation requests for the same SKU cannot silently overwrite one another's update. products is Product Service's single source of truth for catalog data, with sku enforced unique via a MongoDB index.

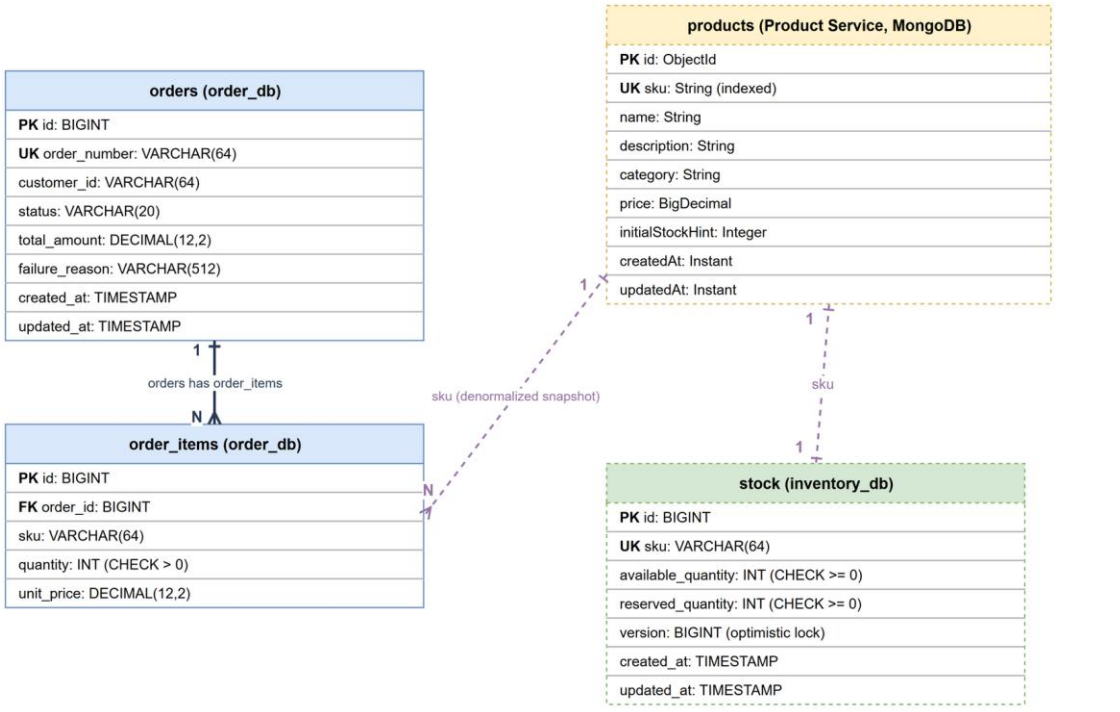
4.4 Relationships

Table 4.01: Foreign Key Relationships Across the Order and Inventory Schemas

Parent	Child	Key	Description
orders	order_items	order_items.order_id → orders.id	Enforced FK, ON DELETE CASCADE. Composition within order_db.
products (Product Svc)	stock (Inventory Svc)	stock.sku ↔ products.sku	Logical only – not an enforced FK, since the two tables live in different databases (database-per-service).
products (Product Svc)	order_items (Order Svc)	order_items.sku ↔ products.sku	Logical only – order_items stores a denormalized sku/unit_price snapshot rather than a live reference.

Table 4.02: Cardinality of Relationships

Entity A	Entity B	Cardinality	Enforced?
orders	order_items	1 : N	Yes (DB)
products	stock	1 : 1	No (app)
products	order_items	1 : N	No (app)



Solid border/edge = enforced FK (same database). Dashed border/edge = different service/database — logical reference only, by SKU, enforced at the application layer (database-per-service). Cardinality labels (1 / N) are shown at each end of every relationship, matching Table 4.02 in the report (1 : N, 1 : 1).

Figure 4.01: Entity-Relationship Diagram – Order and Inventory Schemas

Chapter 5 — Feature Development Process

5.1 Introduction

The Place Order feature (FS-01) is the system's core feature and the one with the most engineering depth, since it has to coordinate a write to the Order Service's own database with a synchronous call to a separate service it does not control. This section walks through its request flow, the resilience engineering built around the Inventory Service call, the compensation logic that keeps data consistent when that call fails partway through an order, and the performance characteristics of the feature.

5.2 Request Flow

A client issues POST /api/orders with a JSON payload containing a customerId and a list of line items (sku, quantity, unitPrice). The request is routed by the API Gateway to OrderController, which delegates to OrderServiceImpl.placeOrder(). For each line item, placeOrder() calls InventoryGateway.reserveStock(sku, quantity) – a Spring 6.1 HTTP Interface client (@HttpExchange, backed by a load-balanced RestClient) that resolves inventory-service through Eureka and issues POST /api/inventory/{sku}/reserve. If every line item reserves successfully, the Order and its OrderItems are persisted with status CONFIRMED inside a single @Transactional method, and an OrderPlacedEvent is published to the order-placed-events Kafka topic, which the Notification Service consumes asynchronously.

5.3 Resilience Engineering

The call to the Inventory Service is wrapped with a Resilience4j circuit breaker and retry, both named "inventoryService" and configured as follows: the circuit breaker uses a count-based sliding window of 10 calls, requires at least 5 calls before computing a failure rate, opens at a 50% failure rate, stays open for 10 seconds, and then allows 3 trial calls in the half-open state; the retry makes up to 3 attempts with a 500ms wait between them. Both are explicitly configured to ignore InsufficientStockException, because a customer ordering more than is in stock is a normal business outcome, not an infrastructure fault – it must not count against the circuit breaker's failure rate, and it must not be retried.

Getting this right required fixing a subtle defect during development: the original fallback method caught every exception the resilience-wrapped call could throw and re-wrapped it, uniformly, as an InventoryServiceUnavailableException before re-throwing. That re-wrapping hid the original exception's type from Resilience4j's own ignore-exceptions check on the @Retry annotation, so a legitimate stock-insufficiency rejection was retried three times with a 500ms backoff before finally surfacing to the client – functionally correct, but wasteful and slow for a case that should fail immediately. The fix was to make the fallback inspect the throwable first and, specifically for InsufficientStockException, re-throw it unchanged so the real type remains visible to the ignore-exceptions check; every other failure is still wrapped as InventoryServiceUnavailableException. This was confirmed with the circuit breaker's own actuator metrics: after the fix, a stock-insufficiency rejection shows exactly one buffered call and zero failed calls, instead of three retried calls.

5.4 Saga-Lite Compensation

Because an order can contain multiple line items and each is reserved with a separate call to the Inventory Service, a failure on item N leaves items 1..N-1 already reserved. OrderServiceImpl tracks every SKU and quantity it has successfully reserved within the current request and, if a later item fails with InsufficientStockException or InventoryServiceUnavailableException, calls InventoryGateway.releaseStockBestEffort() for each of them before persisting the order with status FAILED and a recorded failure reason. This is a deliberately scoped, order-local compensation – a "saga-lite" – rather than a dedicated saga-orchestration framework, which is an appropriate and explainable trade-off for a system with these few downstream steps; a production system with many more steps in the chain would likely need a dedicated orchestrator instead.

Chapter 6 — Deployment Flow

6.1 Deployment Overview

The current deployment target is a single Docker Compose file that brings up the entire stack – all six application services, Keycloak, Kafka, and each service's own datastore – with one command, which is appropriate for a capstone project that needs to be reproducible on any reviewer's machine without a cloud account. Each service is built into its own image from the shared multi-module Maven build, and every inter-service dependency (a service's database, Kafka, Keycloak, and the Discovery Server) is expressed as a Compose healthcheck-gated dependency, so a service does not start accepting traffic until what it depends on is actually ready, not merely started.

6.2 Container Topology

The Compose topology consists of the Discovery Server (Eureka); Keycloak, currently running with its embedded H2 database in development mode; the API Gateway; the Product Service with its own MongoDB container; the Order Service with its own MySQL container (order_db); the Inventory Service with its own MySQL container (inventory_db); the Notification Service; and a Kafka broker that the Order Service produces to and the Notification Service consumes from. All services register with, and discover one another through, the Discovery Server, and all external traffic enters through the API Gateway.

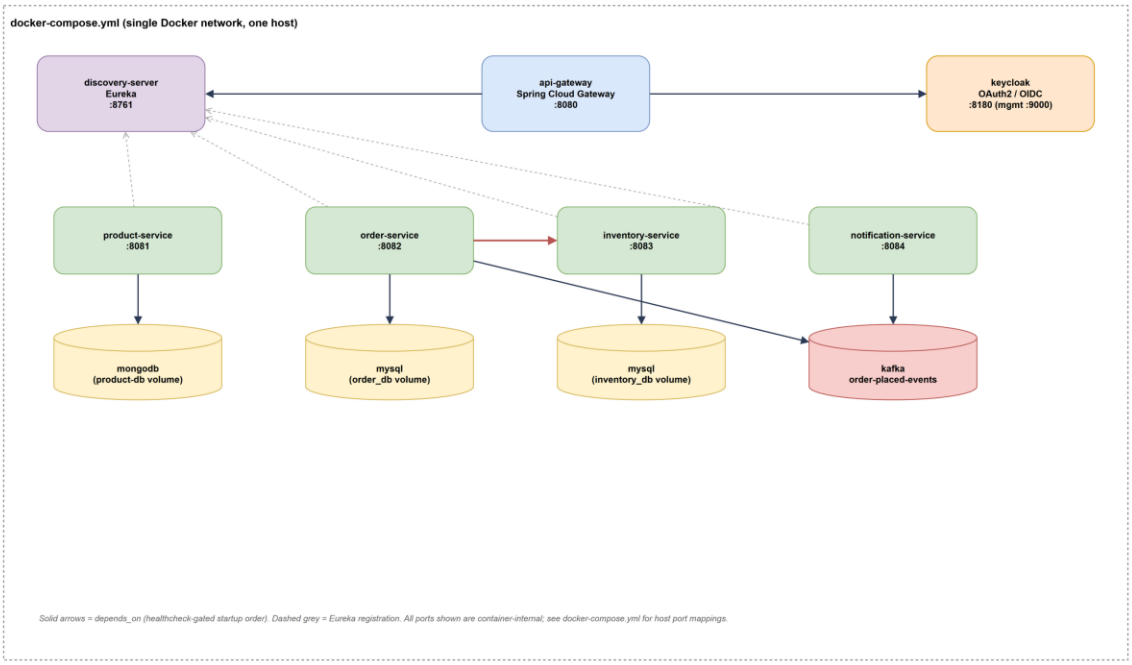


Figure 6.01: Deployment Architecture (Docker Compose)

6.3 Health Checks and Service Discovery

Two deployment issues surfaced during development and were fixed at the Compose level rather than in application code. First, every service's Eureka client URI needed to carry the Discovery Server's Basic Auth credentials, because the Discovery Server's own security configuration requires authentication on every path, including the heartbeat endpoint other services call; the fix embeds the credentials directly in each service's EUREKA_URI. Second, the Discovery Server's own healthcheck used wget flags unsupported by the BusyBox wget shipped in the base image; the fix switched to BusyBox-compatible user:pass@host URL-embedded credentials. A similar issue affected Keycloak's healthcheck: Keycloak 24 and later serve /health/ready on a separate management port (9000) rather than the main port, so the healthcheck was pointed at the correct port.

6.4 Cloud Deployment Path (Future Scope)

The system is not yet deployed to a public cloud. A natural next step, consistent with the project report template's expected AWS-based deployment flow, would be: containers pushed to Amazon ECR and run on ECS/EKS (or EC2 instances) inside a dedicated VPC; Security Groups restricting inbound traffic to the API Gateway only; the MySQL databases

migrated to RDS and Keycloak repointed at an RDS-backed (not embedded H2) persistent store; MongoDB migrated to Atlas or DocumentDB; and an Application Load Balancer or Elastic Beanstalk environment in front of the API Gateway.

Chapter 7 — Technologies Used

7.1 Technology Stack Overview

The platform is built entirely on the Spring ecosystem for application logic, with a deliberately heterogeneous set of datastores and a message broker chosen to match each service's actual access pattern, rather than standardizing on a single database technology across all six services.

7.2 Spring Boot and Spring Cloud

Spring Boot 3.3 provides the application runtime, auto-configuration, and embedded server for all six services. Spring Cloud layers service-discovery (Netflix Eureka) and edge-routing (Spring Cloud Gateway) on top, so that services can be added, moved, or scaled without any client needing a hard-coded address.

7.3 Keycloak and OAuth2/OIDC

Keycloak issues OAuth2/OIDC access tokens, and the API Gateway validates them as an OAuth2 resource server before any request reaches a backend service, centralizing authentication at a single choke point rather than duplicating token validation in every service.

7.4 MongoDB

MongoDB backs the Product Service's catalog. A document model suits a catalog where different product categories may eventually carry different attributes, without requiring a schema migration for every new attribute.

7.5 MySQL and Flyway

MySQL backs the Order Service and Inventory Service, where the data is strongly relational (orders to line items, stock counts with constraints) and benefits from ACID transactions and CHECK constraints. Flyway versions every schema change as a numbered SQL migration, so the schema can be rebuilt deterministically in any environment.

7.6 Apache Kafka

Kafka decouples order confirmation from customer notification: the Order Service publishes an order-placed event and returns to the caller immediately, while the Notification Service consumes that event independently, so a slow or temporarily unavailable notification channel cannot slow down checkout.

7.7 Resilience4j

Resilience4j provides the circuit breaker and retry that wrap the Order Service's call to the Inventory Service, as detailed in Chapter 5, and exposes live circuit-breaker state through Spring Boot Actuator for observability.

7.8 Spring 6.1 HTTP Interface Clients (RestClient)

The Order Service calls the Inventory Service through a declarative @HttpExchange interface backed by a load-balanced RestClient, Spring 6.1's successor to OpenFeign for this project; it removes an extra third-party dependency while keeping the same declarative, interface-based calling style.

7.9 Docker Compose and Testcontainers

Docker Compose orchestrates the full stack for local development and demonstration (Chapter 6). Testcontainers spins up real, disposable MySQL/MongoDB/Kafka containers during the automated test suite, so integration tests exercise real infrastructure instead of mocks, without requiring a persistent test environment.

7.10 OpenAPI / Swagger

springdoc-openapi generates an interactive Swagger UI and OpenAPI document for each service directly from its controller annotations, giving every service a browsable, always-current API reference.

7.11 Technology Summary

Table 7.01: Technology Stack Summary

Technology	Purpose	Application in This Project

Spring Boot 3.3 / Spring Cloud 2023.0.3	Application runtime, auto-configuration	Runtime for all six services
Netflix Eureka	Service discovery	Discovery Server; registration/lookup for all services
Spring Cloud Gateway	Edge routing, resource server	API Gateway; single entry point, JWT validation
Keycloak (OAuth2/OIDC)	Identity and access management	Token issuance; validated by the API Gateway
MongoDB	Document database	Product catalog storage
MySQL / Flyway	Relational database, schema versioning	order_db and inventory_db storage and migrations
Apache Kafka	Event streaming	order-placed-events topic between Order and Notification Service
Resilience4j	Circuit breaker and retry	Guards the Order → Inventory Service call
Spring RestClient / HTTP Interface	Declarative HTTP client	InventoryGateway, load-balanced via Eureka
Docker / Docker Compose	Containerization, orchestration	Runs the full stack locally with one command
Testcontainers	Integration testing	Real disposable MySQL/MongoDB/Kafka in automated tests
springdoc-openapi / Swagger UI	API documentation	Interactive API docs per service

Chapter 8 — Conclusion

8.1 Conclusion

This project set out to demonstrate that a realistic e-commerce order-processing capability can be decomposed into independently deployable microservices without sacrificing the consistency and resilience a monolithic implementation would get almost for free. The six-service architecture, resilience-wrapped inter-service call, saga-lite compensation, event-driven notification, and centralized authentication built here show that these properties can be achieved deliberately, with each engineering decision – the database-per-service boundary, the circuit breaker and retry configuration, the ignore-exceptions distinction between a business rejection and an infrastructure fault – traceable to a specific failure mode the architecture would otherwise be exposed to.

8.2 Key Takeaways

- Decomposing a system by business capability (Product, Order, Inventory, Notification) is only half the work; the other half is deciding, for every cross-service call, what happens when that call is slow, fails, or is legitimately rejected.
- A circuit breaker's ignore-exceptions setting only removes an exception from the failure-rate calculation; it does not, by itself, stop a fallback from hiding that exception's type from a co-located retry policy – the two concerns have to be designed together.
- The database-per-service pattern moves referential integrity for cross-service references (like a SKU shared between the catalog and inventory) from the database layer to the application layer; this is a legitimate and necessary trade-off, but it has to be made consciously, not discovered accidentally.
- Centralizing authentication at the API Gateway, rather than in every service, is both simpler to implement correctly and simpler to audit.

8.3 Practical Applications

The patterns demonstrated in this project are not specific to e-commerce. Service decomposition by business capability, resilience engineering around synchronous inter-service calls, event-driven decoupling of a slow or optional downstream step, and centralized OAuth2 authentication at an API gateway are directly applicable to warehouse and fulfillment systems, booking and reservation platforms, payment-processing pipelines, and any other domain where a single user-facing action (placing an order, booking a seat, authorizing a payment) depends on multiple independently owned backend capabilities that must stay consistent with one another despite running as separate, independently failing processes.

8.4 Limitations

- The API Gateway currently authenticates Postman/test traffic using the OAuth2 Resource Owner Password Credentials (ROPC / Direct Access Grants) flow for convenience; this flow is deprecated in OAuth 2.1 and RFC 9700 and is not an appropriate production authentication flow for a browser or mobile client.
- Keycloak currently persists its realm and user data to its embedded H2 database rather than an external, durable database, so identity data does not survive the Keycloak container's data volume being removed.
- No formal load/performance benchmarking has been carried out yet; the circuit breaker and retry configuration are reasoned about from their configured thresholds, not yet validated under sustained synthetic load.
- The saga-lite compensation in the Order Service is intentionally scoped to a single order's own line items; it is a correct and explainable solution at this scale, but it would not extend cleanly to a workflow with many more downstream steps.
- The system is deployed only via Docker Compose on a single host today; it has not yet been deployed to a public cloud (see Section 6.4).

8.5 Future Scope

- Replace the ROPC flow with Authorization Code + PKCE for user-facing clients and Client Credentials for service-to-service calls, most likely via a Backend-for-Frontend

/ token-relay pattern at the API Gateway, so that the client application never handles the user's raw credentials.

- Move Keycloak from its embedded H2 database to an external, MySQL-backed persistent store, so identity data survives container recreation independently of application data.
- Carry out formal load testing of the Place Order feature (Section 5.5) and tune the circuit breaker/retry thresholds against measured, rather than assumed, latency and failure-rate data.
- Deploy the stack to a public cloud (Section 6.4), with managed databases, a managed Kafka service, and auto-scaling for the Order and Inventory Services specifically, since those two see the most load during checkout.
- Add distributed tracing (for example, Micrometer Tracing with Zipkin or an OpenTelemetry collector) so a single order's path across the Gateway, Order Service, Inventory Service, and Kafka can be followed as one trace instead of being reconstructed from separate service logs.

References

[1] VMware / Broadcom, “Spring Boot Reference Documentation.” [Online]. Available: https://docs.spring.io/spring-boot/

[2] VMware / Broadcom, “Spring Cloud Gateway Reference Documentation.” [Online]. Available: https://docs.spring.io/spring-cloud-gateway/

[3] Netflix, “Eureka Wiki.” [Online]. Available: https://github.com/Netflix/eureka/wiki

[4] Resilience4j, “Resilience4j Documentation.” [Online]. Available: https://resilience4j.readme.io/

[5] Apache Software Foundation, “Apache Kafka Documentation.” [Online]. Available: https://kafka.apache.org/documentation/

[6] Red Hat, “Keycloak Documentation.” [Online]. Available: https://www.keycloak.org/documentation

[7] Internet Engineering Task Force, “RFC 9700: Best Current Practice for OAuth 2.0 Security.” [Online]. Available: https://www.rfc-editor.org/rfc/rfc9700

[8] MongoDB Inc., “MongoDB Manual.” [Online]. Available: https://www.mongodb.com/docs/manual/

[9] Oracle, “MySQL 8.0 Reference Manual.” [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[10] Redgate, “Flyway Documentation.” [Online]. Available: https://documentation.red-gate.com/flyway

[11] Docker Inc., “Docker Compose Documentation.” [Online]. Available: https://docs.docker.com/compose/

[12] Testcontainers, “Testcontainers for Java Documentation.” [Online]. Available: https://java.testcontainers.org/

[13] VMware / Broadcom, “REST Clients – Spring Framework Reference, RestClient and HTTP Interface Clients.” [Online]. Available: https://docs.spring.io/spring-framework/reference/integration/rest-clients.html

[14] Project source repository. [Online]. Available: https://github.com/amolsbrahme/ecommerce-order-management